Individual reflection: randomized quickselect

Ethan Fernandez, Programming Assignment 2, Advanced Algorithms

Worst case

Quickselect's worst case is $\Theta(n^2)$, and it happens when the random pivot keeps landing near the ends of the window. If every pivot were the smallest key, each round would remove one element and the comparisons would add up to about $n^2/2$. What I find interesting is that no input triggers this on distinct keys. The trigger is a sequence of unlucky coin flips, and since each pivot lands in the middle half of the window with probability ½, a long run of bad pivots becomes exponentially unlikely. Over 2,000 pivot seeds on one input of 10^6 keys, my comparisons ranged from 2.1 to 11.8 per element, so the worst run did about 2.4 times the average work, where a quadratic run would need hundreds of thousands per element. The one input that does trigger quadratic behaviour is all-equal keys under Lomuto partitioning, where every pivot has the same value and randomness cannot help.

Hardest bug and design decision

The algorithm itself was quite easy to implement as a short loop, so my hardest bug was a subtle one rather than a logic error. In my Lomuto loop I wrote a[i] <= pivot directly instead of going through less(), which meant the counted build would have missed every comparison in that loop. The tests could not see it because the answers were still correct, so it would have silently corrupted my comparison counts. I fixed it by writing !less(pivot, a[i]) and added a test requiring at least $n - 1$ counted comparisons, then checked the test by temporarily putting the bug back and watching it fail.

The hardest design decision was choosing between Lomuto and three-way partitioning. Lomuto is the textbook version and the simplest to write, but on all-equal input it sends every key to the left, so each round removes only the pivot and the cost is quadratic for every seed, which my duplicates experiment measured as a comparison slope of 2.00. Three-way partitioning fixes this but adds 51 to 62% more comparisons on random input. I kept Lomuto selectable to measure that cost, and it turned out to be only 15% more time at $n = 10^3$ and nothing at 10^7.

What I would do differently

I would research partitioning schemes properly before writing either algorithm. I chose Lomuto because it was the version in the GeeksforGeeks implementation I started from, without thinking about how median-of-medians would use the same partition code, and the duplicates problem only showed up during testing. Looking more closely at Hoare-style partitioning would also have been worthwhile, since nth_element uses it and makes about 2.7 comparisons per element where my quickselect makes 5.3.

What I now understand

I now understand why randomness does not protect against duplicates. A random pivot randomises which position is chosen, not which value, so it defends against bad orderings of the input but not against inputs where many keys share a value. I also understand why quickselect was immune to the batching artefact in my first harness. When a batch held identical copies of one input, the deterministic algorithms made the same branches on every copy and the branch predictor learned

them, making them look three to five times faster. Quickselect drew new pivots on every copy, so there was no pattern to learn.